

Sumador Signado de Alto Rendimiento (CSA)

v1.0.0 — 04 de Noviembre de 2025

José Rodrigo Mata Gómez, José Ramón Estrada Manriquez, Jaime Isaac Navarro Martínez

Diseño de Sistemas Digitales I

Índice

1. Introducción	3
2. Marco Teórico	4
2.1. Bloques Fundamentales	4
2.2. Sumador con acarreo de propagación (RCA)	4
2.3. Anticipación de acarreo y prefijo paralelo	4
2.3.1. Sumador de anticipación de acarreo (CLA)	4
2.3.2. Kogge-Stone	5
2.4. Reducción de la propagación del acarreo	5
2.4.1. Salto del acarreo (CSkA)	5
2.4.2. Selección del acarreo (CSIA)	6
2.5. Optimizaciones de lógica y circuito	6
2.5.1. Sumador Ling	6
2.5.2. Cadenas de acarreo de Manchester	6
3. Estado del Arte	8
3.1. De la teoría a la práctica	8
3.1.1. Familia de sumadores Knowles	8
3.1.2. Un ejemplo de industria: Sumador Han-Carlson	8
3.1.3. Un ejemplo en silicio: Intel	8
4. Desarrollo	9
4.1. Implementación RCA	9
4.2. Implementación CSkA	9
4.3. Implementación sumador de Ling	9
4.4. Implementación del sumador Kogge Stone Adder	10
4.4.1. Pre-procesamiento	10
4.4.2. Red para cálculo de acarreo de grupo	10
4.4.3. Cálculo de Acarreos Finales y Suma	11
4.5. Microarquitecturas del Diseño Modular	12
5. Resultados	13
5.1. Comparativa de Rendimiento	13
5.2. Resultado kogge Stone	13

6. Conclusiones	14
Apéndice A: RTL del Half Adder (HA)	16
Apéndice B: Microarquitectura del Half Adder (HA)	17
Apéndice C: RTL del Full Adder (FA)	18
Apéndice D: Microarquitectura del Full Adder (FA)	19
Apéndice E: RTL del Ripple Carry Adder (RCA) de N bits	20
Apéndice F: Microarquitectura del Ripple Carry Adder (RCA)	21
Apéndice G: Estructura del Carry Skip Adder (CSA) de 16 bits	22

Índice de figuras

1. Microarquitectura del Half Adder.	17
2. Microarquitectura del Full Adder construido con dos Half Adders y una compuerta OR. . . .	19
3. Diagrama de bloques de un Ripple Carry Adder (RCA) de 4 bits. Muestra la conexión en cascada de los Full Adders (FA) y la propagación en serie del acarreo.	21
4. Diagrama de bloques general del Adder segmentado en 4 bits.	22
5. Microarquitectura Kogge Stone Adder para 4-bits. La etapa de preprocesamiento ocurre simultáneamente en todos los bits, mientras que el cálculo de acarreo pasa por 3 bloques de procesamiento antes de llegar a la etapa de posprocesamiento.	26

Índice de cuadros

1. Comparativa de rendimiento de los bloques base post-síntesis.	13
2. Características de funcionamiento Sumador Tipo Kogge Stone	13

1. Introducción

El diseño de circuitos aritméticos eficientes es fundamental en la ingeniería de sistemas digitales, especialmente en unidades de procesamiento (CPU, DSP) donde la suma es una operación recurrente y crítica para el rendimiento. En este proyecto, el objetivo principal es el diseño e implementación de un sumador binario signado de alto rendimiento (signed adder), buscando minimizar el retardo de propagación (critical path delay) para maximizar la frecuencia de operación (f_{Max}).

La representación de números signados se implementa utilizando el complemento a dos, lo que permite que la operación de suma se realice con la misma lógica de un sumador sin signo. Sin embargo, el desafío clave en la suma de N bits radica en el acarreo (carry), cuya propagación en serie a través de las etapas del Sumador de Acarreo por Propagación (Ripple Carry Adder, RCA) limita severamente la velocidad.

Para superar esta limitación, se ha propuesto una arquitectura de sumador que combina el Ripple Carry Adder segmentado con lógica de salto de acarreo (Carry Skip Adder, CSA). Esta técnica permite que el acarreo de entrada pueda saltar a través de bloques de bits cuando se cumplen ciertas condiciones de propagación, reduciendo el tiempo de la ruta crítica y logrando una mejora significativa en el rendimiento.

2. Marco Teórico

2.1. Bloques Fundamentales

El medio sumador y el sumador completo son los componentes básicos de todas las arquitecturas de sumadores complejos. Son los circuitos lógicos básicos responsables de realizar la suma binaria en bits individuales.

- **Half Adder (HA):** Circuito combinacional que suma dos bits (A y B), produciendo una Suma (S) y un Acarreo de Salida (C_{out}).

$$S = A \oplus B$$

$$C_{out} = A \cdot B$$

- **Full Adder (FA):** Circuito que suma tres bits de entrada (A , B , y C_{in}), generando una Suma (S) y un Acarreo de Salida (C_{out}). Es el bloque fundamental para los sumadores multi-bit. Sus ecuaciones son:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

La capacidad del sumador completo para gestionar un acarreo de entrada (C_{in}) permite la conexión en cascada de múltiples sumadores completos para crear sumadores de N bits, lo que forma la base del sumador por propagación de acarreo (ripple carry adder, RCA), que es la estructura base que todos los sumadores de alta velocidad buscan mejorar.

2.2. Sumador con acarreo de propagación (RCA)

El RCA se construye a partir de una cadena de sumadores completos: cada bit (i) calcula una suma (S_i) y un acarreo (C_{i+1}) a partir de las entradas (A_i, B_i) y el acarreo (C_i).

La ruta crítica es la cadena lineal de cálculos de acarreo; el retraso en el peor de los casos crece linealmente con el ancho de bits (n), es decir, el retrasos de puerta ($O(n)$).

2.3. Anticipación de acarreo y prefijo paralelo

2.3.1. Sumador de anticipación de acarreo (CLA)

El concepto de anticipación de acarreo utiliza señales de generación/propagación por bit y un pequeño conjunto de ecuaciones combinatorias para calcular los acarreos antes de la etapa de suma, en lugar de esperar a la propagación de la onda. Para un grupo de bits, la ecuación central es

$$C_{i+1} = G_i + P_i C_i,$$

donde las señales auxiliares se definen como

- **Generar (G_i):** Indica si se genera un acarreo en esa posición.

$$G_i = A_i \cdot B_i$$

- **Propagar (P_i):** Indica si un acarreo entrante se propagará a la siguiente posición.

$$P_i = A_i \oplus B_i$$

y al expandir esto repetidamente, se puede expresar (C_k) como una función booleana del acarreo inicial (C_0) y los pares (G, P) de los bits menos significativos.

Una implementación práctica de dos niveles divide un sumador de (n) bits en grupos: cada etapa de 1 bit calcula (G_i, P_i), una unidad de anticipación de grupo calcula el acarreo de grupo a partir del (G, P) del grupo y el acarreo entrante, y una pequeña ondulación dentro de cada grupo calcula las sumas. Ejemplo del acarreo $i = 2$:

$$C_2 = G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

Esto reduce el peor retraso de acarreo a un pequeño número constante de niveles de puerta por agrupación más los niveles de anticipación utilizados en la implementación.

Las primeras implementaciones modernas de anticipación de acarreo (Carry-Lookahead Adder, CLA) se asocian con Weinberger y Smith [1], que aplicaron el álgebra de generación/propagación a la lógica práctica; CLA es el antecesor de todas las variantes de prefijo paralelo.

2.3.2. Kogge-Stone

El sumador Kogge-Stone (KSA) [2] es un sumador de prefijo paralelo que construye acarreo con una profundidad lógica mínima y una salida muy baja por nodo mediante el uso de un árbol binario de operadores de prefijo (generar/propagar). En un sumador Kogge-Stone de base 2, el cálculo del prefijo se realiza en $(\log_2 n)$ etapas, en las que, en la etapa (k), cada nodo fusiona la información de generación/propagación de distancias (2^{k-1}) separadas. El resultado es que cada acarreo se produce después de $(\log_2 n)$ etapas de prefijo, lo que da una profundidad asintótica $(O(\log n))$ con una pequeña constante.

La desventaja es una complejidad y un área de cableado muy elevadas, lo que puede crear congestión en el enrutamiento y un mayor número total de transistores. Kogge-Stone es omnipresente en los sumadores de CPU de alto rendimiento, donde la velocidad es primordial y el nodo del proceso puede tolerar el coste del cableado y el área.

2.4. Reducción de la propagación del acarreo

2.4.1. Salto del acarreo (CSkA)

El sumador con salto de acarreo (Carry Skip, CSA) acorta la ruta de acarreo en el peor de los casos al permitir que el acarreo "salte" sobre bloques de bits que propagarán el acarreo. Cada bloque calcula una señal de propagación de bloque:

$$P_{group} = \bigwedge_{i \in block} P_i = P_i \cdot P_{i+1} \cdots P_{i+n-1}$$

Si llega el acarreo entrante y ($P_{group} = 1$), entonces el acarreo puede reenviarse al siguiente bloque en una sola etapa de multiplexación de salto, en lugar de propagarse a través de todos los sumadores completos del bloque. En la arquitectura, el CSkA es similar al RCA dentro de los bloques, pero inserta lógica de salto para elegir entre el resultado de propagación (si el bloque no se propaga completamente) o reenviar directamente el acarreo entrante.

La ventaja clave es que, para muchos patrones de entrada realistas, el acarreo a menudo salta varios bloques y, al elegir adecuadamente los tamaños de los bloques, el retraso en el peor de los casos puede reducirse de $(O(n))$ a aproximadamente $(O(\sqrt{n}))$ cuando se ajustan los tamaños de los bloques.

El diseño de salto de acarreo es más ligero en área que un CLA completo porque la lógica de salto de bloque es compacta en comparación con las unidades de anticipación, pero no elimina completamente las dependencias de acarreo. Las formalizaciones originales y las variantes prácticas se publicaron a principios de la década de 1960 y se han perfeccionado repetidamente para implementaciones de baja potencia y alta velocidad [3].

2.4.2. Selección del acarreo (CSIA)

El sumador con selección de acarreo intercambia área por latencia al calcular dos sumas candidatas (y acarreo) para cada bloque en paralelo: una asumiendo que el acarreo del bloque es 0 y otra asumiendo que es 1. Una vez que se conoce el acarreo real (del bloque anterior), un multiplexor elige el resultado del bloque precalculado correcto.

El retraso no es acumulativo. Mientras el bloque i calcula su acarreo, el bloque $i + 1$ ya precalcula ambos resultados posibles. La ruta crítica se convierte simplemente en la cadena de multiplexores que seleccionan las respuestas.

El artículo de Bedrij de 1962 [4] formalizó la técnica e informó de las grandes ganancias de velocidad posibles cuando se dispone de mucha área para paralelizar los acarreo. El enfoque de selección de acarreo es una de las formas más sencillas de obtener una latencia casi constante por bloque a costa de duplicar aproximadamente el cálculo por bloque, y de necesitar una mayor área total para todo el sumador.

2.5. Optimizaciones de lógica y circuito

2.5.1. Sumador Ling

El sumador Ling es una variante de la familia prefijo paralelo que reescribe las ecuaciones de acarreo para reducir la complejidad del cálculo de la suma final y, en ocasiones, reducir la profundidad lógica. Ling [5] observó que la recurrencia del acarreo puede transformarse algebraicamente para simplificar algunas de las operaciones de combinación de prefijos; en la práctica, la reformulación de Ling a menudo conduce a una lógica más simple para las etapas más significativas o a menos pilas de transistores en las implementaciones CMOS.

$$H_i = C_i + C_{i-1}$$

Al reducir la complejidad de la ecuación de acarreo, Ling permite que las puertas lógicas conmuten más rápido. Elimina eficazmente un nivel de retardo lógico de la ruta crítica de la cadena de acarreo.

El sumador Ling se utiliza principalmente como optimización de las implementaciones de anticipación/prefijo para reducir el retraso o la profundidad lógica en determinadas etapas, y varios diseños modernos adaptan las transformaciones de Ling dentro de árboles de prefijos más grandes.

2.5.2. Cadenas de acarreo de Manchester

La cadena de acarreo Manchester [6] es una técnica de hardware utilizada en VLSI para acelerar la propagación del acarreo mediante la disposición de redes de transistores de forma que exista una ruta rápida para

la propagación del acarreo, a la vez que se utiliza lógica compacta para la evaluación de generación/propagación. Un modo de operación de implementar cadenas de Manchester con transistores interruptores:

- Si P_i (propagar) es alto, el interruptor se cierra, conectando el nodo de acarreo directamente con el nodo de acarreo de entrada.
- Si G_i (generar) es alto, el nodo de acarreo de salida se conecta a VDD (1).
- Si K_i (eliminar) es alto, el nodo de acarreo de salida se conecta a tierra (0).

La cadena reduce los niveles lógicos y el cableado al compartir la lógica entre celdas de bits adyacentes (tomando nodos intermedios del dispositivo que forma el acarreo más significativo), implementando eficazmente una ruta cableada rápida que sintetiza la información del acarreo con mayor rapidez que las puertas lógicas ingenuas. Es ideal cuando se requiere regularidad en el diseño y un bajo número de transistores, y donde la tecnología de proceso soporta las estructuras de transistores requeridas.

3. Estado del Arte

3.1. De la teoría a la práctica

En los microprocesadores modernos (como los de la serie Intel Core, AMD Zen o Apple Silicon), casi nunca se encuentra un sumador Kogge-Stone puro o un sumador CLA puro. Esto se debe a la ingeniería de cableado. En tecnologías submicrónicas profundas, el retardo suele estar dominado por los cables que conectan las puertas, no por las puertas en sí. Un sumador Kogge-Stone puro presenta una congestión de cableado masiva que consume demasiada energía y área.

En cambio, las ALU modernas utilizan arquitecturas híbridas que equilibran la profundidad lógica y la distribución del cableado.

3.1.1. Familia de sumadores Knowles

Simon Knowles propuso en un artículo [7] una familia parametrizada de topologías de sumadores de prefijos que generaliza muchos de los árboles clásicos (Kogge-Stone, Sklansky, Brent-Kung, Ladner-Fischer, etc.) mediante la introducción de parámetros de diseño que compensan la profundidad, el recuento de nodos, la salida y el cableado. En lugar de una única topología fija, la construcción de Knowles permite seleccionar parámetros que producen redes sumadoras en puntos intermedios del espacio de diseño.

En la familia de sumadores, todos los sumadores de prefijo paralelo utilizan el álgebra de propagar y generar (P, G) . La contribución de Knowles consistió en definir un método de construcción para el objetivo deseado: elegir una segmentación/bloqueo y, a continuación, aplicar etapas de combinación de prefijos de forma controlada para poder limitar la dispersión y el número de nodos, al tiempo que se mantiene una profundidad baja. La ventaja del enfoque de Knowles es la flexibilidad. Para un objetivo determinado (biblioteca de celdas, nodo de proceso, retardo de cableado, fan-out máximo), se pueden seleccionar parámetros que minimicen el retardo en el peor de los casos, al tiempo que se mantiene el cableado y el recuento de nodos manejables.

3.1.2. Un ejemplo de industria: Sumador Han-Carlson

El sumador Han-Carlson es un sumador híbrido de prefijo paralelo que mezcla el área reducida de los árboles Brent-Kung con la baja profundidad lógica de los árboles de estilo Kogge-Stone. Se introdujo para los implementadores de VLSI que necesitaban una profundidad casi óptima, pero que también querían limitar la complejidad del cableado y la dispersión de las puertas [8].

En la topología Han-Carlson se calculan los acarreo combinando señales de generación/propagación en un árbol: la cadena más larga de operadores de prefijo es de solo $(O(\log n))$. Esto convierte la latencia del acarreo en el peor de los casos de lineal a logarítmica en ancho de bits. Dado que Han-Carlson conserva las fusiones paralelas similares a las de Kogge en las etapas iniciales, conserva la mayor parte de la ventaja de profundidad de Kogge a la vez que ahorra área.

3.1.3. Un ejemplo en silicio: Intel

Varios artículos de Intel de principios de la década del 2000 describen unidades de lógica aritmética (ALU) y unidades de generación de direcciones (AGU) en silicio [9], [10]. El artículo sobre una AGU afirma explícitamente que el sumador de uno de sus procesadores es un sumador de árbol disperso de 32 bits implementado semidinámicamente para alcanzar un funcionamiento en altas frecuencias (GHz). El segundo artículo de Intel menciona que la ALU implementaba una unidad de suma Han-Carlson para una unidad de procesador de enteros.

4. Desarrollo

En este trabajo se presenta el diseño de 3 implementaciones de sumadores de alto rendimiento. Los diseños se abordan de forma jerárquica y modular. La estructura se basa en optimizar primero los bloques fundamentales (HalfAdder, FullAdder) para luego construir el rcaN como unidad base de segmento, y finalmente el módulo cseaN (Carry Skip Adder de N bits) como el diseño de rendimiento final.

4.1. Implementación RCA

- **Implementación de Bloques Base:** Se diseñó y verificó el HalfAdder y el FullAdder utilizando assign para la lógica combinacional.
- **Implementación RCA Base:** Se implementa el RCA encadenando N módulos FullAdder. Este módulo se utiliza como el bloque interno de suma de los segmentos del CSA.
- **Análisis de Línea Base:** Se realizó la síntesis de los módulos HalfAdder, FullAdder y RCA para medir el WNS (Worst Negative Slack) y la Frecuencia Máxima (F_{MAX}), estableciendo el rendimiento de la arquitectura RCA pura.
- **Optimización:** Actualmente se trabaja en la resolución de problemas de síntesis y la optimización de la lógica del *skip carry* para lograr la máxima velocidad.

4.2. Implementación CSkA

La implementación del módulo cseaN implica:

- Instanciar N módulos RCA.
- Calcular las señales de propagación de grupo (P_{group}) para cada segmento.
- Utilizar una lógica de Salto (AND/OR y Multiplexores) para calcular el acarreo de salida de cada segmento, seleccionando entre la propagación interna del RCA o el acarreo saltado del segmento anterior.

4.3. Implementación sumador de Ling

La formula recursiva esta basada en la tabla de verdad del par (a_i, b_i) . Al examinar los pares de bit, el acarreo se propaga P_i y el acarreo se genera G_i en paralelo. La reformulación de Ling surge al considerar los pares de bits adyacentes anteriores (a_{i+1}, b_{i+1}) , con la que se obtiene un nuevo acarreo recursivo.

Las formulaciones de H_i y H_{i-1} como funciones del par de bits $(i, i-1)$. La señales de H_i y H_{i-1} es generada por a_i, b_i . Esta nueva señal, o nuevo acarreo, solo puede terminar cuando el inhibidor $(a_{i-1} + b_{i-1} = 0)$. En este sentido, H_i es el acarreo y la señal complementaria necesaria para realizar la suma binaria.

Al agrupar todas las H_i se obtiene

$$\begin{aligned} H_i &= a_i \cdot b_i + H_{i-1} \cdot (\bar{a}_{i-1} \cdot b_{i-1} + a_{i-1} \cdot \bar{b}_{i-1} + a_{i-1} \cdot b_{i-1}) \\ &= a_i \cdot b_i + H_{i-1} \cdot (a_{i-1} \cdot b_{i-1}) = G_i + H_{i-1} \cdot P_{i-1} \end{aligned}$$

donde G_i es la nueva señal complementaria, H_{i-1} es la señal complementaria previa, y P_{i-1} es el interruptor habilitador de acarreo de la etapa previa.

Para la implementación del sumador de Ling se toma como referencia el artículo original de Ling [5] con algunas modificaciones:

- Computar la etapa de Generar, Propagar y media suma (G_n, P_n, D_n) .
- Calcular los acarreo con la función de Ling: $H_i = C_i + C_{i-1} = G_i + (P_i \cdot H_{i-1})$.
- Computar la suma completa con los acarreo de la señal de Ling.

Nota: En su artículo original [5] y publicaciones subsecuentes, Ling describe la señal de propagación P_i como la disyunción de un par de bits $(a_i + b_i)$ y no como la disyunción exclusiva del par $(a_i \oplus b_i)$, comúnmente asociada con la familia de arboles de sumadores con prefijo en paralelo, como Kogge Stone.

4.4. Implementación del sumador Kogge Stone Adder

Se busca realizar un sumador que trabaje a la máxima frecuencia posible sin limitaciones de área o consumo y para esto se inicio la búsqueda de una arquitectura en la que se obtenga estos resultados de obtener un sumador que opere a frecuencias muy altas, se observaron diferentes opciones y una de ellas es el sumador de prefijo paralelo tipo Kogge Stone por diferentes características que presenta que lo hace una de las arquitecturas mas rápidas de sumadores que existe.

Este tipo de sumador es considerado como la versión modificada de los sumadores de acarreo de prefijos con enfoque en reducción de tiempo y mejora de rendimiento. El Kogge Stone Adder está diseñado a partir de tres etapas que son el pre-procesamiento, red de acarreo de anticipación y el pos-procesamiento. [11]

Etapas Fundamentales del KSA

El diseño se basa en tres etapas principales:

4.4.1. Pre-procesamiento

Para cada par de entradas A_i y B_i , donde $i \in \{0, 1, \dots, 31\}$, se calculan las señales de Propagación (P_i) y Generación (G_i).

- **Propagación (P_i):** Indica si el acarreo de la posición $i - 1$ se propaga a la posición i .

$$P_i = A_i \oplus B_i$$

- **Generación (G_i):** Indica que se genera un acarreo local en la posición i .

$$G_i = A_i \cdot B_i$$

4.4.2. Red para cálculo de acarreo de grupo

Esta es la red logarítmica ($\log_2 N = 5$ etapas) que calcula la Propagación de Grupo ($P_{k:i}$) y la Generación de Grupo ($G_{k:i}$) para bloques de bits cada vez mayores. La operación de combinación fundamental es:

Operador de Prefijo: Combina dos subgrupos adyacentes, $k : j + 1$ y $j : i$.

- **Propagación de Grupo ($P_{k:i}$):**

$$P_{k:i} = P_{k:j+1} \cdot P_{j:i}$$

- **Generación de Grupo ($G_{k:i}$):**

$$G_{k:i} = G_{k:j+1} + (P_{k:j+1} \cdot G_{j:i})$$

Cálculos por Etapa

Las etapas ($m = 0$ a 4) se caracterizan por el retardo $\Delta = 2^m$ entre las entradas combinadas.

Etapas (m)	Retardo ($\Delta = 2^m$)	Ecuación Recurrente ($i \geq \Delta$)
0	1	$G_{i:i-1} = G_i + (P_i \cdot G_{i-1})$
1	2	$G_{i:i-2} = G_{i:i-1} + (P_{i:i-1} \cdot G_{i-2:i-3})$
2	4	$G_{i:i-4} = G_{i:i-1} + (P_{i:i-1} \cdot G_{i-4:i-5})$
3	8	$G_{i:i-8} = G_{i:i-1} + (P_{i:i-1} \cdot G_{i-8:i-9})$
4	16	$G_{i:i-16} = G_{i:i-1} + (P_{i:i-1} \cdot G_{i-16:i-17})$

4.4.3. Cálculo de Acarreos Finales y Suma

Acarreos de Entrada (C_i)

El acarreo de entrada a la posición i (C_i) se calcula utilizando las señales de grupo generadas en la red de prefijos y el acarreo inicial C_{in} .

- El acarreo de entrada al bit 0 es el acarreo inicial:

$$C_0 = C_{in}$$

- El acarreo de entrada a cualquier posición i es:

$$C_i = G_{i-1:0} + (P_{i-1:0} \cdot C_{in})$$

Donde $G_{i-1:0}$ y $P_{i-1:0}$ son las señales de Propagación y Generación para el rango completo de bits 0 a $i - 1$, obtenidas de la última etapa relevante.

- El acarreo de salida final es C_{32} :

$$C_{32} = G_{31:0} + (P_{31:0} \cdot C_{in})$$

Cálculo de la Suma (S_i)

Finalmente, el bit de suma S_i es el XOR de la Propagación local P_i y el acarreo de entrada C_i .

$$S_i = P_i \oplus C_i$$

4.5. Microarquitecturas del Diseño Modular

- **HA:** Compuesto por una compuerta XOR y una compuerta AND.
- **FA:** Se puede construir a partir de dos HalfAdders y una compuerta OR.
- **RCA:** La microarquitectura es una cadena de FullAdders.
- **CSkA:** Microarquitectura similar al RCA, con bloques P_{group} que verifica que todos los bits de un grupo sean 1. De ser cierto, un multiplexor pasa el acarreo del grupo al siguiente bloque.
- **Ling:** Primera etapa calcula paralelamente (G_n, P_n, D_n) con compuertas AND, OR, y XOR. La segunda etapa calcula la señal de Ling H_n con compuertas OR. La última etapa computa la suma y el último acarreo con compuertas XOR.
- **Kogge-Stone:** Realización en etapas. Cada etapa requiere

Los diagramas y esquemas de los bloques fundamentales se presentan en los apéndices. Los RTL de las 4 implementaciones se presentan también en el apéndice.

5. Resultados

5.1. Comparativa de Rendimiento

La siguiente tabla registra las métricas de rendimiento tras la síntesis de los bloques fundamentales, comparando la implementación RTL pura con el uso de primitivas.

Módulo	Implementación	WNS [ns]	f _{Max} [MHz]
Half Adder (HA)	RTL Puro (SV)	-4.917	100.83
Half Adder (HA)	Con Primitivas	-4.917	100.83
Full Adder (FA)	RTL Puro (SV)	-4.933	100.67
Full Adder (FA)	Con Primitivas	-4.933	100.67
Ripple Carry Adder (RCA) <i>N</i> bits	RTL Puro (SV)	-1.279	159.26
Pipelined Adder (PLA) 4 bits	RTL Final (SV)	-1.140	318.52

Cuadro 1: Comparativa de rendimiento de los bloques base post-síntesis.

5.2. Resultado kogge Stone

Bits	Power Consumtion	Max Frequency
4	0.227W	1.434GHz
8	0.233W	729.4MHz
16	0.243W	479.616MHz
32	0.269W	437.254MHz

Cuadro 2: Características de funcionamiento Sumador Tipo Kogge Stone

6. Conclusiones

El desarrollo de los sumadores signados y no signados permitió cumplir con los objetivos principales: implementar arquitecturas funcionales en RTL, compararlas contra versiones basadas en primitivas y evaluar su impacto en el retardo crítico y la frecuencia máxima de operación. Se verificó que los módulos fundamentales (HA, FA, RCA) funcionan correctamente y que el uso de pipeline en el PLA mejora el rendimiento esperado, aunque aún existe margen para una mayor optimización del flujo de síntesis y del balance entre etapas.

Entre los aprendizajes técnicos destacan la importancia de analizar el camino crítico desde la etapa RTL, comprender cómo la propagación del acarreo afecta la velocidad del sistema y reconocer por qué los sumadores constituyen un bloque limitante en datapaths complejos. Asimismo, se observó que el tratamiento del bit de signo requiere especial atención, pues influye en la extensión de signo, el manejo de overflow y la forma en que el sintetizador infiere la lógica interna.

Como posibles mejoras, se propone expandir la arquitectura hacia sumadores lookahead o carry-skip, integrar opciones de paralelismo o pipeline más profundas y evaluar su impacto sobre el rendimiento y área en FPGA. También sería valioso explorar versiones parametrizables más robustas, orientadas a aplicaciones DSP o procesadores embebidos donde estos sumadores constituyen la base de unidades aritméticas de mayor escala.

Finalmente, el uso de SystemVerilog demostró ser ventajoso frente a Verilog al ofrecer tipado más estricto, manejo claro de señales signadas, módulos parametrizables y estructuras de control más expresivas. Estas características reducen ambigüedades en la inferencia durante síntesis y facilitan la construcción de arquitecturas aritméticas escalables y mantenibles.

Referencias

- [1] Arnold Weinberger y JL Smith. «A logic for high-speed addition». En: *Nat. Bur. Stand. Circ* 591 (1958), págs. 3-12.
- [2] Peter M. Kogge y Harold S. Stone. «A Parallel Algorithm for the Efficient Solution of a General Class of Recurrence Equations». En: *IEEE Transactions on Computers* C-22.8 (ago. de 1973), págs. 786-793. ISSN: 0018-9340. DOI: 10.1109/TC.1973.5009159. URL: <http://ieeexplore.ieee.org/document/5009159/> (visitado 08-12-2025).
- [3] M. Lehman y N. Burla. «Skip Techniques for High-Speed Carry-Propagation in Binary Arithmetic Units». En: *IEEE Transactions on Electronic Computers* EC-10.4 (dic. de 1961), págs. 691-698. ISSN: 0367-7508. DOI: 10.1109/TEC.1961.5219274. URL: <http://ieeexplore.ieee.org/document/5219274/> (visitado 08-12-2025).
- [4] O. J. Bedrij. «Carry-Select Adder». En: *IRE Transactions on Electronic Computers* EC-11.3 (jun. de 1962), págs. 340-346. ISSN: 0367-9950. DOI: 10.1109/IRETELC.1962.5407919. URL: <http://ieeexplore.ieee.org/document/5407919/> (visitado 08-12-2025).
- [5] Huei Ling. «High Speed Binary Parallel Adder». En: *IEEE Transactions on Electronic Computers* EC-15.5 (oct. de 1966), págs. 799-802. ISSN: 0367-7508. DOI: 10.1109/PGEC.1966.264571. URL: <http://ieeexplore.ieee.org/document/4038889/> (visitado 08-12-2025).
- [6] T. Kilburn, D.B.G. Edwards y D. Aspinall. «Parallel addition in digital computers: a new fast 'carry' circuit». en. En: *Proceedings of the IEE - Part B: Electronic and Communication Engineering* 106.29 (sep. de 1959), págs. 464-466. ISSN: 2054-0418. DOI: 10.1049/pi-b-2.1959.0316. URL: <https://digital-library.theiet.org/content/journals/10.1049/pi-b-2.1959.0316> (visitado 08-12-2025).
- [7] S. Knowles. «A family of adders». En: *Proceedings 14th IEEE Symposium on Computer Arithmetic (Cat. No.99CB36336)*. Adelaide, SA, Australia: IEEE Comput. Soc, 1999, págs. 30-34. ISBN: 9780769501161. DOI: 10.1109/ARITH.1999.762825. URL: <http://ieeexplore.ieee.org/document/762825/> (visitado 08-12-2025).
- [8] Tackdon Han y David A. Carlson. «Fast area-efficient VLSI adders». En: *1987 IEEE 8th Symposium on Computer Arithmetic (ARITH)*. Mayo de 1987, págs. 49-56. DOI: 10.1109/ARITH.1987.6158699. URL: <https://ieeexplore.ieee.org/document/6158699> (visitado 08-12-2025).
- [9] S. Vangal et al. «5-GHz 32-bit integer execution core in 130-nm dual-V/sub T/ CMOS». en. En: *IEEE Journal of Solid-State Circuits* 37.11 (nov. de 2002), págs. 1421-1432. ISSN: 0018-9200. DOI: 10.1109/JSSC.2002.803944. URL: <http://ieeexplore.ieee.org/document/1046084/> (visitado 08-12-2025).
- [10] S. Mathew et al. «A 4-GHz 130-nm address generation unit with 32-bit sparse-tree adder core». en. En: *IEEE Journal of Solid-State Circuits* 38.5 (mayo de 2003), págs. 689-695. ISSN: 0018-9200. DOI: 10.1109/JSSC.2003.810056. URL: <http://ieeexplore.ieee.org/document/1196212/> (visitado 08-12-2025).
- [11] Reshma Mary James y Ajith Ravindran. «Review of Full Adder Performance Analysis Using Kogge Stone Adder and Magnetic Tunnel Junction». En: *2018 4th International Conference on Devices, Circuits and Systems (ICDCS)*. Coimbatore: IEEE, mar. de 2018, págs. 84-90. ISBN: 9781538634769. DOI: 10.1109/ICDCSyst.2018.8605064. URL: <https://ieeexplore.ieee.org/document/8605064/> (visitado 10-12-2025).

Apéndices

Apéndice A: RTL del Half Adder (HA)

```
1 module half_adder(  
2     input logic clk,  
3     input logic arst_n,  
4     input logic a,  
5     input logic b,  
6     output logic s,  
7     output logic c  
8 );  
9 //internal signals  
10 logic s_comb;  
11 logic c_comb;  
12 logic s_reg;  
13 logic c_reg;  
14 //combinational logic  
15 assign s_comb = a ^ b;  
16 assign c_comb = a & b;  
17 //sequential logic  
18 always_ff (posedge clk, negedge arst_n) begin  
19     if(!arst_n) begin  
20         s_reg <= 1'b0;  
21         c_reg <= 1'b0;  
22     end else begin  
23         s_reg <= s_comb;  
24         c_reg <= c_comb;  
25     end  
26 end  
27 //outputs  
28 assign s = s_reg;  
29 assign c = c_reg;  
30 endmodule
```

Listing 1: Half Adder (HA) RTL Code

Apéndice B: Microarquitectura del Half Adder (HA)

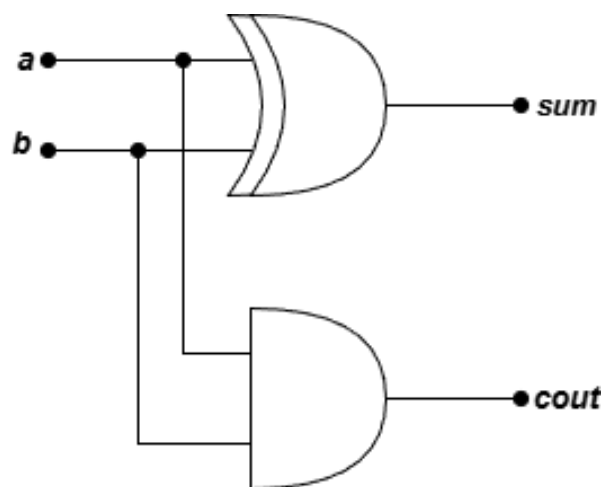


Figura 1: Microarquitectura del Half Adder.

Apéndice C: RTL del Full Adder (FA)

```
1 module full_adder(  
2     input logic a,  
3     input logic b,  
4     input logic cin,  
5     output logic s,  
6     output logic cout  
7 );  
8  
9 assign cout = ((a ^ b) & cin) | (a & b);  
10 assign s = a ^ b ^ cin;  
11  
12 endmodule
```

Listing 2: Full Adder (FA) RTL Code

Apéndice D: Microarquitectura del Full Adder (FA)

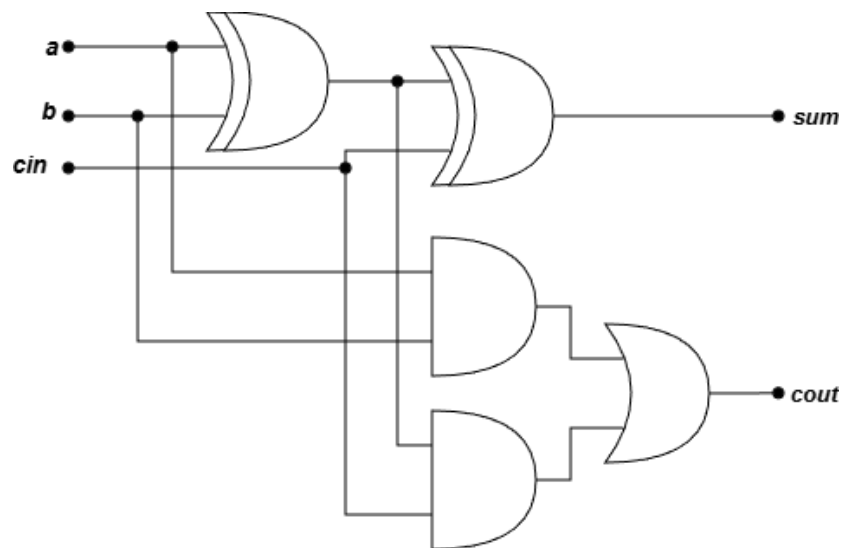


Figura 2: Microarquitectura del Full Adder construido con dos Half Adders y una compuerta OR.

Apéndice E: RTL del Ripple Carry Adder (RCA) de N bits

```
1 module ripple_adder #(parameter N = 128)(
2     input logic signed [N-1:0] a,
3     input logic signed [N-1:0] b,
4     input logic cin,
5     output logic signed [N-1:0] s,
6     output logic overflow
7 );
8
9 //internal signal for carry propagation
10 logic cout;
11 logic [N-1:0] carry;
12 assign carry[0] = cin; //carry in assigned to first full adder
13
14 genvar i;
15 generate
16     for(i = 0; i < N; i++) begin : full_adder_gen
17         full_adder FA (
18             .a(a[i]),
19             .b(b[i]),
20             .cin(carry[i]),
21             .s(s[i]),
22             .cout(carry[i+1])
23         );
24     end
25 endgenerate
26 //outputs
27 assign cout = carry[N]; // final carry out
28 assign overflow = carry[N-1] ^ carry[N]; //if the result is incorrect
29                                     //carry[MSB] ^ carry[LSB] = 1'b1
30
31 endmodule
```

Listing 3: Ripple Carry Adder (RCA) N-bit RTL Code (rcaN)

Apéndice F: Microarquitectura del Ripple Carry Adder (RCA)

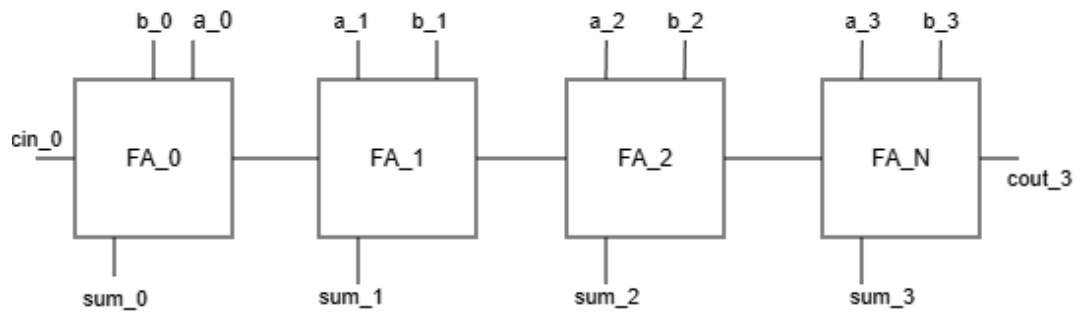


Figura 3: Diagrama de bloques de un Ripple Carry Adder (RCA) de 4 bits. Muestra la conexión en cascada de los Full Adders (FA) y la propagación en serie del acarreo.

Apéndice G: Estructura del Carry Pipelined Adder (PLA) de 4 bits

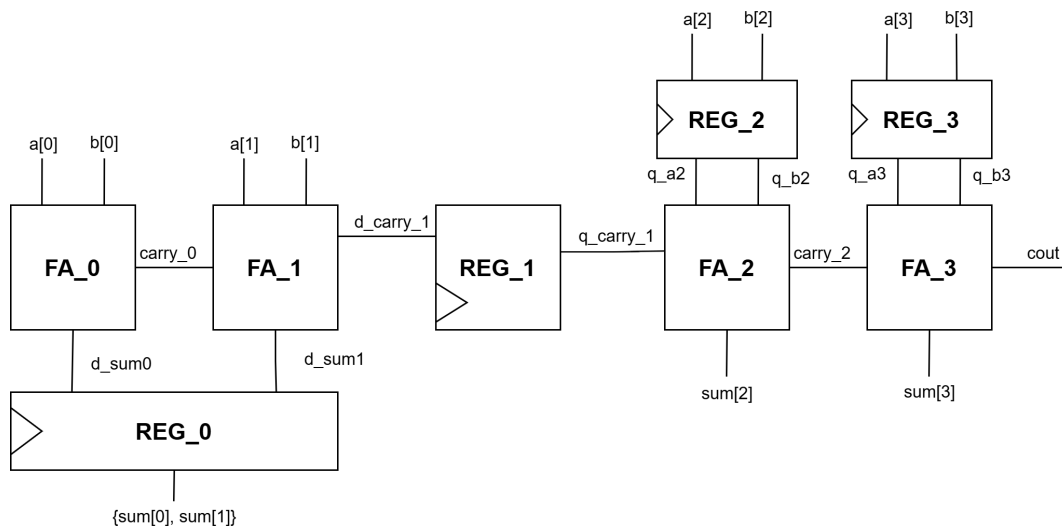


Figura 4: Diagrama de bloques general del Adder segmentado en 4 bits.

Apéndice H: RTL del sumador de Ling

```
1 module ling_adder #(parameter N = 8)(
2   input logic [N-1:0] a,    // First operand
3   input logic [N-1:0] b,    // Second operand
4   input logic      cin,    // Input carry
5   output logic [N-1:0] sum, // sum
6   output logic      cout   // Output carry
7 );
8
9 logic [N-1:0] G; // "Generate" signal
10 logic [N-1:0] P; // "Propagate" signal
11 logic [N-1:0] D; // Mod-2 half-sum
12 logic [N-1:0] L; // Ling signal for carries
13
14 // Step 1: Generate, Propagate, and half-sum
15 always_comb begin
16   for (int i = 0; i < N; i++) begin
17     G[i] = a[i] & b[i];
18     P[i] = a[i] | b[i];
19     D[i] = a[i] ^ b[i];
20   end
21 end
22
23 // Step 2: Ling Logic function
24 assign L[0] = G[0] ^ cin;
25
26 generate
27   for (genvar i = 1; i < N; i++) begin : ling_step
28     assign L[i] = G[i] | (P[i] & L[i-1]);
29   end
30 endgenerate
31
32 // Step 3: Full-sum computation with the carries of the Ling signal
33 generate
34   for (genvar i = 1; i < N; i++) begin : sum_step
35     assign sum[i] = D[i] ^ L[i-1]; // L_{i-1} is the carry of the previous sum
36   end
37 endgenerate
38
39 // Last processing.
40 assign sum[0] = D[0] ^ cin; // LSB of the sum has to consider the input carry
41 assign cout = L[N-1]; // The carry bit is the MSB of the Ling signal.
42 endmodule
```

Listing 4: Ling Adder N-bit RTL Code

Apéndice I: RTL del sumador Kogge-Stone

```
1 module ksa_corregido(  
2     input logic clk,  
3     input logic arst_n,  
4     input logic en,  
5     input logic [7:0] first,  
6     input logic [7:0] second,  
7     input logic carry_in,  
8     output logic [7:0] final_result,  
9     output logic carry_out_final  
10 );  
11 logic [7:0] pro;  
12 logic [7:0] gen;  
13 logic [7:1] pro_step0; //1  
14 logic [7:1] gen_step0;  
15 logic [7:2] pro_step1; //3  
16 logic [7:2] gen_step1;  
17 logic [7:4] pro_step2; //6  
18 logic [7:4] gen_step2;  
19 logic [8:0] carries;  
20 logic [7:0] result;  
21  
22 logic carry_in_reg;  
23 reg_carry carry_reg(  
24     .clk(clk),  
25     .arst_n(arst_n),  
26     .en(1'b1),  
27     .in(carry_in),  
28     .out(carry_in_reg)  
29 );  
30  
31 genvar i;  
32 generate  
33     for(i=0; i<8; i++)begin:initialstep  
34         assign pro[i] = first[i] ^ second[i];  
35         assign gen[i] = first[i] & second[i];  
36     end  
37 endgenerate  
38  
39 // Etapa 0 G', P'  
40 genvar j;  
41 generate  
42     for(j=1; j<8; j++)begin:step0  
43         assign pro_step0[j] = pro[j] & pro[j-1];  
44         assign gen_step0[j] = gen[j] | (pro[j] & gen[j-1]);  
45     end  
46 endgenerate  
47  
48  
49  
50 // Etapa 1 G'', P''  
51  
52 assign pro_step1[2] = pro_step0[2] & pro[0];  
53 assign gen_step1[2] = gen_step0[2] | (pro_step0[2] & gen[0]);  
54  
55 genvar k;  
56 generate  
57     for(k=3; k<8; k++)begin:step1  
58         assign pro_step1[k] = pro_step0[k] & pro_step0[k-2]; //cambio de 1 a 2  
59         assign gen_step1[k] = gen_step0[k] | (pro_step0[k] & gen_step0[k-2]);  
60     end  
61 endgenerate  
62  
63
```

```

64 //Etapa 2 G''', P'''
65
66 assign pro_step2[4] = pro_step1[4] & pro[0];
67 assign gen_step2[4] = gen_step1[4] | (pro_step1[4] & gen[0]);
68
69 assign pro_step2[5] = pro_step1[5] & pro_step0[1];
70 assign gen_step2[5] = gen_step1[5] | (pro_step1[5] & gen_step0[1]);
71
72 genvar l;
73 generate
74     for(l=6; l<8; l++)begin:step2
75         assign pro_step2[l] = pro_step1[l] & pro_step1[l-4];
76         assign gen_step2[l] = gen_step1[l] | (pro_step1[l] & gen_step1[l-4]);
77     end
78 endgenerate
79
80
81 //c0=Cin -0
82 //C1= p0 & c0 | g0 -1
83 //C2= p'1 & c0 | g'1 -2
84 //C3= p''2 & c1 | g''2 -2
85 //C4= p'''3 & c0 | g'''3 -4
86 //C5= p''''4 & c1 | g''''4 -4
87 //C6= p''''5 & c2 | g''''5 -4
88 //C7= p''''6 & c3 | g''''6 -4
89 //C8= p''''7 & c0 | g''''7 -8
90 assign carries[0] = carry_in;
91 assign carries[1] = gen[0] | (pro[0] & carry_in);
92 assign carries[2] = gen_step0[1] | (pro_step0[1] & carry_in);
93 assign carries[3] = gen_step1[2] | (pro_step1[2] & ((pro[0] & carry_in) | gen[0])
94 );
95 assign carries[4] = gen_step1[3] | (pro_step1[3] & carry_in_reg);
96 assign carries[5] = gen_step2[4] | (pro_step2[4] & ((pro[0] & carry_in) | gen[0])
97 );
98 assign carries[6] = gen_step2[5] | (pro_step2[5] & ((pro_step0[1] & carry_in) |
99 gen_step0[1]));
100 assign carries[7] = gen_step2[6] | (pro_step2[6] & ((pro_step1[2] & ((pro[0] &
101 carry_in) | gen[0])) | gen_step1[2]));
102 assign carries[8] = gen_step2[7] | (pro_step2[7] & carry_in);
103 assign carry_out = carries[8];
104
105 genvar m;
106 generate
107     for(m=0; m<8; m++)begin:results
108         assign result[m] = pro[m] ^ carries[m];
109     end
110 endgenerate
111
112 assign carry_out_final = carry_out;
113 assign final_result = result;
114 endmodule

```

Listing 5: Kogge-Stone Adder 8-bit RTL Code

Apéndice J: Microarquitectura Kogge Stone Adder

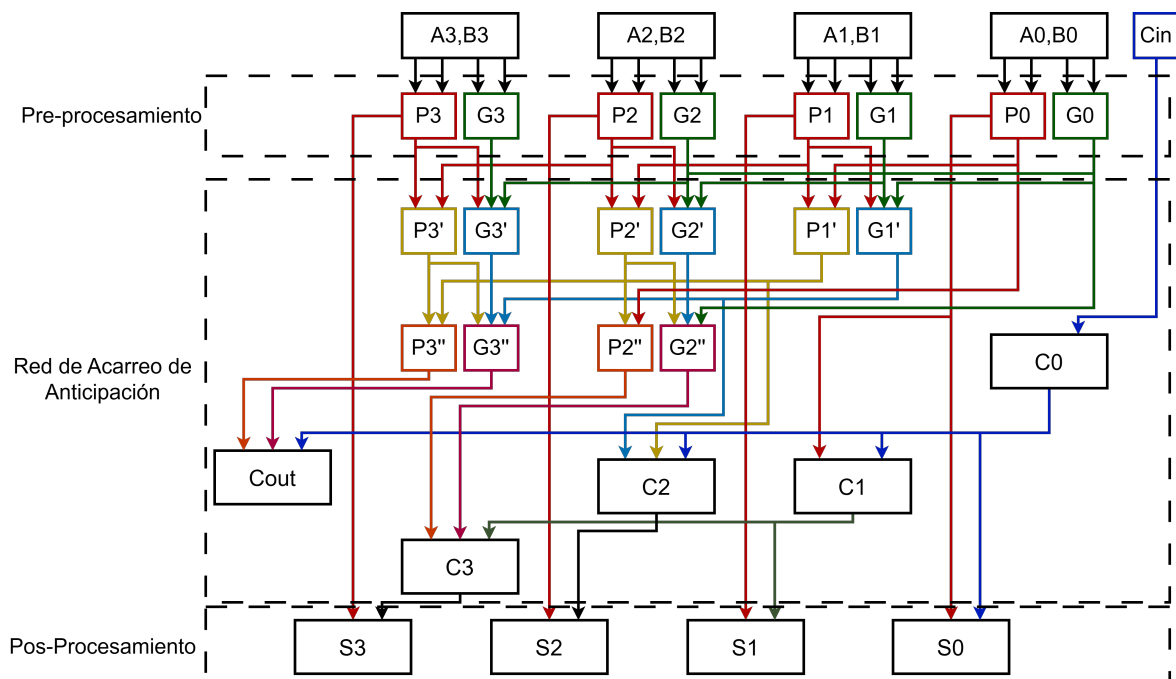


Figura 5: Microarquitectura Kogge Stone Adder para 4-bits. La etapa de preprocesamiento ocurre simultáneamente en todos los bits, mientras que el cálculo de acarreo pasa por 3 bloques de procesamiento antes de llegar a la etapa de posprocesamiento.